NODE.JS

COMPLETE GUIDE

A Comprehensive Backend Development Reference

6 Chapters • 60 Interview Q&As • Production Patterns

Topics Covered

› Node.js Basics & the Event Loop
› Express.js Framework
› Routing & Middleware
› Database Integration (SQL & NoSQL)
› Async Programming (Callbacks, Promises, async/await)
› Error Handling & Production Best Practices

2026 Edition

CHAPTER 1

Node.js Basics & the Event Loop

What Node.js is, how it executes JavaScript, and why the event loop is its superpower

1. What Is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 JavaScript engine. It allows you to run JavaScript on the server — outside the browser — making it possible to build web servers, APIs, CLI tools, real-time applications, and microservices entirely in JavaScript.

Unlike traditional server environments that spawn a new thread for every incoming request (Apache, traditional Java), Node.js uses a single-threaded, non-blocking I/O model. This makes it exceptionally efficient for I/O-heavy workloads like REST APIs, real-time chat, and streaming — but not ideal for CPU-intensive tasks like video encoding or complex mathematical computations.

> *Node.js is not a framework or a library — it is a runtime. It provides the environment (V8 engine + libuv + built-in modules) that lets JavaScript run on your server, your laptop, or inside a Docker container.*

2. Installing Node.js and Your First Server

```
# Check installed version
node --version    # v20.x.x
npm --version     # 10.x.x

# Run a JavaScript file
node app.js

# Start a Node.js REPL (interactive shell)
node
```

```
// hello.js — the simplest possible Node.js HTTP server
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello from Node.js!');
```

```
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});

// Visit http://localhost:3000 in your browser to see the response
```

3. Node.js Architecture — V8, libuv, and the Event Loop

Node.js is built on three core components working together. V8 is Google's JavaScript engine — it compiles JS to machine code for fast execution. libuv is a C++ library that provides the event loop, thread pool, and OS-level async I/O. The Node.js Bindings layer ties these together so your JavaScript can call OS-level operations.

```
// Node.js Architecture (simplified):

// ┌─────────────────────────────────────────┐
// │          Your JavaScript Code           │
// ├─────────────────────────────────────────┤
// │   Node.js APIs (fs, http, path...)      │
// ├─────────────────────────────────────────┤
// │       Node.js Bindings (C++)            │
// ├──────────────────────┬──────────────────┤
// │   V8 Engine          │     libuv        │
// │  (JS execution)      │  (event loop +   │
// │                      │   thread pool)   │
// └──────────────────────┴──────────────────┘

// libuv's thread pool (default 4 threads) handles:
// - File system operations (fs.readFile)
// - DNS lookups
// - Crypto operations (bcrypt)
// Network I/O (TCP/UDP) uses OS async primitives directly,
// NOT the thread pool — this is why Node handles many connections cheaply
```

4. The Event Loop — In Depth

The event loop is the mechanism that allows Node.js to perform non-blocking I/O despite using a single thread. It continuously checks whether the call stack is empty and, if so, processes callbacks from various queues in a specific order. Understanding this order is critical for writing correct async code.

```
// Event Loop Phases (executed in order every 'tick'):

// 1. timers          — executes setTimeout() and setInterval() callbacks
```

```
// 2. pending I/O     — executes I/O callbacks deferred from previous loop
// 3. idle/prepare    — internal Node.js use only
// 4. poll            — retrieves new I/O events; blocks here if queue is empty
// 5. check           — executes setImmediate() callbacks
// 6. close callbacks — socket.on('close', ...) callbacks

// Between each phase: process.nextTick() queue runs first,
//                     then Promise microtask queue

// Priority order (highest to lowest):
// process.nextTick() > Promise.then() > setImmediate() > setTimeout(0)
```

4.1 Event Loop Execution Order — Live Example

```
console.log('1 — synchronous start');

setTimeout(() => console.log('2 — setTimeout 0ms'), 0);

setImmediate(() => console.log('3 — setImmediate'));

Promise.resolve().then(() => console.log('4 — Promise microtask'));

process.nextTick(() => console.log('5 — process.nextTick'));

console.log('6 — synchronous end');

// Output order:
// 1 — synchronous start
// 6 — synchronous end
// 5 — process.nextTick        ← nextTick queue (runs before everything async)
// 4 — Promise microtask       ← microtask queue
// 2 — setTimeout 0ms          ← timers phase
// 3 — setImmediate            ← check phase
```

5. Non-Blocking I/O — Why Node.js Scales

The key to Node.js performance is non-blocking I/O. When Node.js makes a file read or database query, it does not sit and wait — it registers a callback and immediately moves on to handle other work. When the I/O completes, the callback is placed in the event queue and executed on the next appropriate loop iteration.

```
const fs = require('fs');

// ── BLOCKING (bad in Node.js) ──────────────────────────────
// Synchronous — blocks the entire thread until file is read
// No other requests can be handled during this time!
const data = fs.readFileSync('large-file.txt', 'utf8');
console.log('File content:', data);
```

```
console.log('This runs AFTER file read');

// ── NON-BLOCKING (correct Node.js style) ──────────────────────
// Asynchronous — registers callback and immediately returns
// Other requests CAN be handled while the file is being read
fs.readFile('large-file.txt', 'utf8', (err, data) => {
  if (err) { console.error('Error:', err); return; }
  console.log('File content:', data);
});
console.log('This runs IMMEDIATELY — before file read completes');

// With 1000 concurrent requests:
// Blocking: handles 1 at a time (999 waiting)
// Non-blocking: handles all 1000 concurrently with 1 thread
```

6. Core Built-in Modules

Node.js ships with a rich set of built-in modules. These require no npm install — they are part of the Node.js runtime itself.

```
// ── fs — File System ──────────────────────────────────────────
const fs = require('fs');
const fsPromises = require('fs/promises');  // promise-based API

// Read file (async)
fs.readFile('data.json', 'utf8', (err, content) => {
  if (err) throw err;
  console.log(JSON.parse(content));
});

// Write file (async)
fs.writeFile('output.txt', 'Hello Node!', (err) => {
  if (err) throw err;
  console.log('Written successfully');
});

// Modern: promise-based fs
const content = await fsPromises.readFile('data.json', 'utf8');

// ── path — File Paths ─────────────────────────────────────────
const path = require('path');
path.join(__dirname, 'public', 'index.html');  // OS-safe join
path.extname('file.json');   // '.json'
path.basename('/home/user/file.txt');  // 'file.txt'
path.dirname('/home/user/file.txt');   // '/home/user'

// ── os — Operating System info ────────────────────────────────
const os = require('os');
os.cpus().length;    // number of CPU cores
os.totalmem();       // total RAM in bytes
os.hostname();       // machine hostname
```

```
// ── events — EventEmitter ──────────────────────────────────
const { EventEmitter } = require('events');
const emitter = new EventEmitter();
emitter.on('data', (payload) => console.log('Received:', payload));
emitter.emit('data', { id: 1, name: 'Node.js' });

// ── crypto — Hashing and Encryption ────────────────────────
const crypto = require('crypto');
const hash = crypto.createHash('sha256').update('password').digest('hex');
```

7. CommonJS vs ES Modules

Node.js originally used CommonJS (require/module.exports). ES Modules (import/export) — the browser standard — are now fully supported in Node.js 12+ via .mjs files or by setting 'type': 'module' in package.json.

```
// ── CommonJS (CJS) — traditional Node.js ──────────────────
// math.js
function add(a, b) { return a + b; }
function multiply(a, b) { return a * b; }
module.exports = { add, multiply };

// app.js
const { add, multiply } = require('./math');
console.log(add(2, 3));       // 5

// ── ES Modules (ESM) — modern standard ────────────────────
// math.mjs (or set 'type':'module' in package.json)
export function add(a, b) { return a + b; }
export function multiply(a, b) { return a * b; }
export default { add, multiply };

// app.mjs
import { add } from './math.mjs';
import defaultExport from './math.mjs';

// Key differences:
// CJS: synchronous, dynamic (require() can be inside if blocks)
// ESM: static (imports hoisted), supports top-level await
// ESM cannot use __dirname/__filename directly — use import.meta.url instead
import { fileURLToPath } from 'url';
const __dirname = path.dirname(fileURLToPath(import.meta.url));
```

8. npm — Node Package Manager

```
# Initialise a new project (creates package.json)
npm init -y
```

```
# Install a package (adds to dependencies)
npm install express
npm install express mongoose dotenv   # install multiple

# Install dev dependency
npm install --save-dev nodemon jest

# Install globally
npm install -g typescript

# Run scripts defined in package.json
npm start
npm run dev
npm test

// package.json scripts section
{
  'scripts': {
    'start': 'node server.js',
    'dev':   'nodemon server.js',
    'test':  'jest'
  }
}

# package-lock.json — locks exact dependency versions
# node_modules/    — never commit this to git
# .gitignore should always include: node_modules/
```

Interview Questions & Answers

Q1: What is Node.js and how does it differ from browser JavaScript?

- Node.js is a JavaScript runtime built on Chrome's V8 engine that executes JavaScript on the server — outside the browser
- Browser JS has access to the DOM, window, document, and Web APIs (localStorage, fetch). Node.js has none of these but instead has access to the file system, OS, networking, and process information via its built-in modules
- Node.js uses CommonJS modules (require/module.exports) by default, while browsers use ES Modules (import/export)
- Both environments run the same V8 engine for JavaScript execution — the difference is entirely in the surrounding APIs and global objects available

Q2: What is the Node.js event loop and how does it work?

- The event loop is a loop that continuously checks whether the call stack is empty and, if so, picks up the next callback from its queue to execute
- It has multiple phases executed in order: timers (setTimeout/setInterval), pending I/O, poll (waits for I/O), check (setImmediate), and close callbacks

- Between each phase, Node.js drains two micro-task queues: process.nextTick() first (highest priority), then resolved Promise callbacks
- This design allows Node.js to handle thousands of concurrent connections on a single thread — the thread is never blocked because I/O operations are delegated to libuv and OS APIs

Q3: What is the difference between process.nextTick() and setImmediate()?

- process.nextTick() executes its callback at the end of the current operation — before the event loop moves to the next phase. It runs before any I/O callbacks, before setTimeout, and before setImmediate
- setImmediate() executes in the check phase — after the poll phase completes, meaning after I/O callbacks for the current iteration have run
- process.nextTick() can starve the event loop if called recursively — if you keep calling nextTick inside nextTick, I/O callbacks never get a chance to run
- Use process.nextTick() for ensuring a callback runs before any I/O. Use setImmediate() for deferring work to the next event loop iteration without starving I/O

Q4: What is non-blocking I/O and why is it important in Node.js?

- Non-blocking I/O means that when Node.js initiates an I/O operation (file read, DB query, HTTP request), it does not wait for the result — it registers a callback and immediately continues executing other code
- When the I/O completes, the OS notifies libuv, which places the callback into the event queue. The event loop picks it up when the call stack is empty
- This allows a single-threaded Node.js server to handle thousands of concurrent connections — no request blocks another because the thread is never sitting idle waiting
- Blocking operations (like fs.readFileSync) should be avoided in request handlers because they pause the entire event loop, preventing all other requests from being processed

Q5: What is the difference between CommonJS and ES Modules in Node.js?

- CommonJS (CJS) uses require() and module.exports — synchronous, dynamic loading, available in all Node.js versions, and supports __dirname and __filename
- ES Modules (ESM) use import and export — static (imports are hoisted and analysed at parse time), support top-level await, and use import.meta.url instead of __dirname
- A .js file uses CJS by default unless 'type': 'module' is set in package.json. Files with .mjs extension are always treated as ESM; .cjs are always CJS
- CJS modules cannot directly import ESM modules. ESM can import CJS modules. For new projects, ESM is the modern standard; most libraries still ship CJS for maximum compatibility

Q6: What is libuv and what role does it play in Node.js?

- libuv is a multi-platform C++ library that provides Node.js with the event loop, asynchronous I/O, a thread pool, and OS-level abstractions

- It maintains a thread pool (default 4 threads, configurable via UV_THREADPOOL_SIZE) that handles operations that have no async OS primitive — file system, DNS, and crypto
- Network I/O (TCP, UDP) does NOT use the thread pool — libuv uses OS async primitives (epoll on Linux, kqueue on macOS, IOCP on Windows) for network operations, which is why Node.js can handle thousands of network connections cheaply
- Without libuv, Node.js would need to block the main thread for every I/O operation, destroying the performance advantage of the event loop model

Q7: What is the V8 engine and how does it execute JavaScript?

- V8 is Google's open-source JavaScript engine, written in C++, that compiles JavaScript directly to native machine code rather than interpreting it line by line
- It uses JIT (Just-In-Time) compilation — it starts executing quickly with a baseline compiler, then identifies hot functions and re-compiles them with aggressive optimisations
- V8 manages memory with a garbage collector that automatically frees memory that is no longer referenced, using a generational GC strategy (new space for short-lived objects, old space for long-lived ones)
- The same V8 engine powers Google Chrome — this is why JavaScript written for the browser largely works the same in Node.js

Q8: What is __dirname and __filename in Node.js?

- __dirname is a global variable in CommonJS modules that contains the absolute path of the directory containing the currently executing file
- __filename contains the absolute path of the currently executing file including the filename
- They are essential for building file paths relative to the current module: path.join(__dirname, 'views', 'index.html') — this works regardless of where the process was started from
- In ES Modules, __dirname is not available. Equivalent: const __dirname = path.dirname(fileURLToPath(import.meta.url))

Q9: What is package.json and what are its most important fields?

- package.json is the manifest file for a Node.js project — it describes the project, its dependencies, and available scripts
- name and version are required for packages published to npm. main specifies the entry point file. type: 'module' enables ES Modules by default
- dependencies contains packages required at runtime (express, mongoose). devDependencies contains packages only needed during development (jest, nodemon, typescript)
- scripts defines shorthand commands: npm start runs 'start', npm run dev runs 'dev'. The engines field specifies compatible Node.js and npm version ranges

Q10: Why is Node.js not suitable for CPU-intensive tasks?

- Node.js runs JavaScript on a single thread. CPU-intensive operations (image processing, video encoding, complex maths) block that thread and prevent the event loop from processing any other work
- While the operation runs, all incoming HTTP requests queue up — the server becomes completely unresponsive to everyone until the computation finishes
- Solutions: use Worker Threads (the worker_threads module) to run CPU work on separate threads, use child_process to fork a separate process, or delegate CPU work to a dedicated microservice
- Node.js excels at I/O-bound workloads: REST APIs, real-time apps, proxies, and streaming — where time is spent waiting for databases and networks, not burning CPU cycles

CHAPTER 2

Express.js

The de-facto Node.js web framework — building APIs and web servers the right way

1. What Is Express.js?

Express.js is a minimal, unopinionated web framework for Node.js. It wraps Node's built-in http module and adds a clean API for routing, middleware, request/response handling, and template rendering. It is the most widely used Node.js framework and the foundation of the MEAN/MERN stack.

'Unopinionated' means Express does not dictate how you structure your application, which database to use, or how to validate data. This gives you full control but also means you must make those decisions yourself — or use a more opinionated framework like NestJS that is built on top of Express.

> *Express is to Node.js what jQuery was to the browser — it does not replace Node.js, it makes working with it far more ergonomic by providing routing, middleware chaining, and a cleaner request/response API.*

2. Setting Up an Express Application

```
# Create project and install Express
mkdir my-api && cd my-api
npm init -y
npm install express
npm install --save-dev nodemon   # auto-restart on file changes

# package.json scripts
{
  'scripts': {
    'start': 'node server.js',
    'dev':   'nodemon server.js'
  }
}
```

```
// server.js — minimal Express application
const express = require('express');
const app     = express();
const PORT    = process.env.PORT || 3000;

// Built-in middleware — parse JSON request bodies
```

```
app.use(express.json());

// Built-in middleware — parse URL-encoded form bodies
app.use(express.urlencoded({ extended: true }));

// Basic route
app.get('/', (req, res) => {
  res.json({ message: 'Welcome to my API', version: '1.0' });
});

// Start the server
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

module.exports = app;  // export for testing
```

3. Request and Response Objects

Express enhances Node's raw req and res objects with convenience properties and methods. Understanding them is essential for building any API.

3.1 The Request Object (req)

```
app.post('/products/:id', (req, res) => {
  // URL parameters — /products/42
  console.log(req.params.id);        // '42' (always a string)

  // Query string — /products/42?sort=price&page=2
  console.log(req.query.sort);       // 'price'
  console.log(req.query.page);       // '2'

  // Request body — requires express.json() middleware
  console.log(req.body.name);        // from JSON body
  console.log(req.body.price);

  // Headers
  console.log(req.headers['authorization']);  // 'Bearer ...'
  console.log(req.get('Content-Type'));        // 'application/json'

  // Other useful properties
  console.log(req.method);           // 'POST'
  console.log(req.path);             // '/products/42'
  console.log(req.url);              // '/products/42?sort=price'
  console.log(req.ip);               // client IP address
  console.log(req.hostname);         // 'example.com'
});
```

3.2 The Response Object (res)

```
app.get('/demo', (req, res) => {
  // Send JSON response (sets Content-Type: application/json automatically)
  res.json({ success: true, data: [] });

  // Send with custom status code
  res.status(201).json({ message: 'Created' });
  res.status(404).json({ error: 'Not found' });

  // Send plain text
  res.send('Hello World');

  // Set headers before sending
  res.set('X-Custom-Header', 'value');
  res.set({ 'Cache-Control': 'no-store', 'X-RateLimit': '100' });

  // Redirect
  res.redirect('/new-path');           // 302 by default
  res.redirect(301, '/permanent');     // 301 permanent

  // Send a file
  res.sendFile(path.join(__dirname, 'public', 'index.html'));

  // Download a file
  res.download('./report.pdf', 'Monthly-Report.pdf');

  // End without body (204 No Content)
  res.status(204).end();
});
```

4. Building a Complete REST API

Here is a full CRUD REST API for a products resource — the standard pattern for real-world Express applications. In production, the data layer would be replaced with a real database service.

```
// routes/products.js
const express = require('express');
const router  = express.Router();

// In-memory store (replace with DB in production)
let products = [
  { id: 1, name: 'Laptop',   price: 999,  category: 'electronics' },
  { id: 2, name: 'Keyboard', price: 79,   category: 'electronics' },
];
let nextId = 3;

// GET /api/products — list all (with optional ?category filter)
router.get('/', (req, res) => {
  const { category } = req.query;
  const result = category
    ? products.filter(p => p.category === category)
```

```
    : products;
  res.json({ success: true, count: result.length, data: result });
});

// GET /api/products/:id — get single product
router.get('/:id', (req, res) => {
  const product = products.find(p => p.id === Number(req.params.id));
  if (!product) return res.status(404).json({ error: 'Product not found' });
  res.json({ success: true, data: product });
});

// POST /api/products — create product
router.post('/', (req, res) => {
  const { name, price, category } = req.body;
  if (!name || !price) {
    return res.status(400).json({ error: 'name and price are required' });
  }
  const product = { id: nextId++, name, price: Number(price), category };
  products.push(product);
  res.status(201).json({ success: true, data: product });
});

// PUT /api/products/:id — replace entire product
router.put('/:id', (req, res) => {
  const idx = products.findIndex(p => p.id === Number(req.params.id));
  if (idx === -1) return res.status(404).json({ error: 'Not found' });
  products[idx] = { id: products[idx].id, ...req.body };
  res.json({ success: true, data: products[idx] });
});

// PATCH /api/products/:id — partial update
router.patch('/:id', (req, res) => {
  const product = products.find(p => p.id === Number(req.params.id));
  if (!product) return res.status(404).json({ error: 'Not found' });
  Object.assign(product, req.body);
  res.json({ success: true, data: product });
});

// DELETE /api/products/:id — delete product
router.delete('/:id', (req, res) => {
  const idx = products.findIndex(p => p.id === Number(req.params.id));
  if (idx === -1) return res.status(404).json({ error: 'Not found' });
  products.splice(idx, 1);
  res.status(204).end();
});

module.exports = router;

// server.js — mount the router
const productRouter = require('./routes/products');
app.use('/api/products', productRouter);
```

5. Environment Variables with dotenv

```
# .env — never commit this file to git!
PORT=3000
NODE_ENV=development
DB_URI=mongodb://localhost:27017/myapp
JWT_SECRET=supersecretkey123

# .gitignore
.env
node_modules/
```

```
// Load environment variables at app entry point (top of server.js)
require('dotenv').config();   // npm install dotenv

const PORT    = process.env.PORT    || 3000;
const DB_URI  = process.env.DB_URI;
const SECRET  = process.env.JWT_SECRET;
const IS_PROD = process.env.NODE_ENV === 'production';

// Best practice: validate required env vars on startup
const REQUIRED = ['DB_URI', 'JWT_SECRET'];
REQUIRED.forEach(key => {
  if (!process.env[key]) {
    console.error(`FATAL: Missing required env var: ${key}`);
    process.exit(1);
  }
});
```

6. Serving Static Files and Template Engines

```
// Serve static files from a 'public' directory
app.use(express.static(path.join(__dirname, 'public')));
// Files in /public are served at http://localhost:3000/filename.ext

// Custom static path prefix
app.use('/static', express.static('public'));
// http://localhost:3000/static/logo.png

// ── Template engine (EJS example) ──────────────────────
// npm install ejs
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

app.get('/dashboard', (req, res) => {
  res.render('dashboard', {
    title:    'Dashboard',
    user:     { name: 'Gowtham', role: 'admin' },
    products: [{ name: 'Laptop', price: 999 }]
  });
});

// views/dashboard.ejs
```

Interview Questions & Answers

Q1: What is Express.js and why is it used with Node.js?

- Express is a minimal, unopinionated web framework for Node.js that adds routing, middleware, and a cleaner req/res API on top of Node's built-in http module
- Without Express, you would manually parse URLs, request bodies, set headers, and manage routing with if-else chains — Express abstracts all of that into a clean, chainable API
- It is unopinionated — it does not dictate project structure, ORM choice, or validation library — giving full flexibility
- Express is the foundation of many full-stack frameworks (NestJS, Sails.js) and is used in production by companies including PayPal, IBM, and Uber

Q2: What is the difference between app.use() and app.get()?

- app.get(path, handler) registers a route handler that responds only to HTTP GET requests at the specified path
- app.use(path, handler) mounts middleware that runs for ALL HTTP methods at the specified path (and all sub-paths). The path argument is optional — app.use(handler) runs for every request
- Middleware registered with app.use() passes control to the next middleware by calling next(). Route handlers with app.get() typically end the response with res.json() or res.send()
- Order matters — app.use() and app.get() handlers are evaluated in the order they are registered

Q3: What is the difference between req.params, req.query, and req.body?

- req.params — contains named route parameters from the URL path. For route /users/:id, accessing /users/42 gives req.params.id === '42'
- req.query — contains key-value pairs from the URL query string. For /products?sort=price&page=2, req.query.sort === 'price' and req.query.page === '2'
- req.body — contains the parsed request body. Requires express.json() middleware for JSON bodies or express.urlencoded() for HTML form data. Not populated for GET requests
- All three values are strings by default — always convert numeric params: Number(req.params.id) or parseInt(req.query.page, 10)

Q4: What does res.json() do and how is it different from res.send()?

- res.json(obj) serialises the JavaScript object to a JSON string, sets Content-Type: application/json automatically, and sends the response

- res.send(body) sends a response with automatic type detection: a string sets Content-Type: text/html; a Buffer sets application/octet-stream; an object calls res.json() internally
- res.json() is preferred for APIs because it always sets the correct Content-Type header and handles null and undefined values correctly (send() would not send a body for null)
- Neither send nor json can be called after the response has already been sent — doing so throws 'Cannot set headers after they are sent' — always return after sending

Q5: How do you structure a large Express application?

- Separate routes into individual router files using express.Router() — one file per resource (products.js, users.js, orders.js)
- Mount routers in server.js with app.use('/api/products', productRouter) — this prefixes all routes in the router with /api/products
- Separate controllers (request handlers) from routes — routes define paths and call controller functions; controllers contain the actual logic
- Further separate business logic into services and data access into repository/model files — routes should contain no business logic, only delegation

Q6: What is nodemon and why is it used in development?

- nodemon is a development tool that monitors your project files and automatically restarts the Node.js process when any file changes
- Without it, you must manually stop (Ctrl+C) and restart node server.js after every code change — tedious during active development
- Install as a dev dependency: npm install --save-dev nodemon, then use it in the dev script: nodemon server.js
- In production, never use nodemon — use a process manager like PM2 (pm2 start server.js) which handles restarts, clustering, logging, and monitoring

Q7: How do you use environment variables in a Node.js/Express application?

- Store configuration in a .env file (PORT, DB_URI, JWT_SECRET) and load it with require('dotenv').config() at the very top of your entry file
- Access values via process.env.VARIABLE_NAME. Always provide fallbacks for non-critical variables: const PORT = process.env.PORT || 3000
- Never commit the .env file to version control — add it to .gitignore. Provide a .env.example file with placeholder values for other developers
- In production, set environment variables directly in the hosting platform (Heroku config vars, AWS Parameter Store, Docker --env-file) rather than using a .env file

Q8: How do you enable CORS in an Express application?

- CORS (Cross-Origin Resource Sharing) is a browser security policy that blocks requests from a different origin (domain/port) unless the server explicitly allows it

- Install the cors package: npm install cors. Use app.use(cors()) to allow all origins, or configure it: app.use(cors({ origin: 'https://myapp.com' }))
- For fine-grained control: app.use(cors({ origin: ['http://localhost:4200', 'https://prod.com'], methods: ['GET','POST','PUT','DELETE'], credentials: true }))
- Apply cors() before any route definitions so all routes have CORS headers. For specific routes only: router.get('/public', cors(), handler)

Q9: What is the difference between app.listen() and http.createServer()?

- app.listen(PORT, callback) is an Express shorthand that internally calls http.createServer(app).listen(PORT, callback)
- Using http.createServer(app) explicitly is needed when you want to add WebSocket support (ws library) — both HTTP and WebSocket servers share the same underlying server instance
- const server = http.createServer(app); const io = new SocketIO(server); server.listen(PORT) — this pattern enables both REST API and real-time WebSocket on the same port
- For HTTPS: use https.createServer({ key, cert }, app).listen(443) — app.listen() does not support SSL directly

Q10: How do you chain multiple handlers to a single route in Express?

- Express route methods accept multiple handler functions as arguments or an array: app.get('/path', handler1, handler2, handler3)
- Each handler receives (req, res, next) and must call next() to pass control to the next handler, or send a response to end the chain
- This is used for applying route-specific middleware: app.post('/admin', authenticate, authorise('admin'), createUserHandler)
- Returning early with return next() (or return res.json(...)) is critical — forgetting to return after calling next() causes both handlers to run and can cause 'headers already sent' errors

CHAPTER 3

Routing & Middleware

Structure your API with modular routers and build reusable middleware for auth, logging, and validation

1. How Express Routing Works

Express routing matches incoming HTTP requests to handler functions based on the HTTP method (GET, POST, PUT, DELETE) and the URL path. Requests are matched against registered routes in the order they were defined — the first matching route handles the request.

```
const express = require('express');
const app = express();

// HTTP method routes
app.get('/users',     handler);   // GET  /users
app.post('/users',    handler);   // POST /users
app.put('/users/:id', handler);   // PUT  /users/42
app.patch('/users/:id', handler); // PATCH
app.delete('/users/:id', handler);// DELETE

// app.all() — matches ALL HTTP methods
app.all('/secret', (req, res) => {
  res.status(405).json({ error: 'Method not allowed' });
});

// Route chaining — same path, different methods
app.route('/books')
  .get((req, res)  => res.json(books))
  .post((req, res) => { books.push(req.body); res.status(201).json(req.body); });

app.route('/books/:id')
  .get((req, res)    => { /* find one */ })
  .put((req, res)    => { /* replace */ })
  .delete((req, res) => { /* remove  */ });
```

2. Route Parameters, Wildcards, and Patterns

```
// Named parameters — :paramName
app.get('/users/:userId/posts/:postId', (req, res) => {
  const { userId, postId } = req.params;
  res.json({ userId, postId });
  // GET /users/5/posts/10 → { userId: '5', postId: '10' }
});
```

```
// Optional parameter — :format?
app.get('/files/:filename.:format?', (req, res) => {
  const { filename, format = 'json' } = req.params;
  res.json({ filename, format });
  // /files/data.csv   → { filename: 'data', format: 'csv' }
  // /files/data       → { filename: 'data', format: 'json' }
});

// Wildcard — * matches anything
app.get('/cdn/*', (req, res) => {
  res.json({ path: req.params[0] });
  // /cdn/images/logo.png → { path: 'images/logo.png' }
});

// Regex route
app.get(/\/products\/\d+/, (req, res) => {
  res.send('Matched numeric product ID only');
});

// Catch-all 404 — MUST be registered LAST
app.use((req, res) => {
  res.status(404).json({ error: `Cannot ${req.method} ${req.path}` });
});
```

3. Express Router — Modular Routing

express.Router() creates a mini-application that handles a subset of routes. It is the standard way to organise a large API — each resource gets its own router file, and routers are mounted in the main app with a base path prefix.

```
// routes/users.js
const express    = require('express');
const router     = express.Router();
const userCtrl   = require('../controllers/user.controller');
const { authenticate, authorise } = require('../middleware/auth');

// All routes here are relative to whatever path this router is mounted at
router.get('/',           authenticate, userCtrl.getAll);
router.get('/:id',        authenticate, userCtrl.getById);
router.post('/',          authenticate, authorise('admin'), userCtrl.create);
router.put('/:id',        authenticate, userCtrl.update);
router.delete('/:id',     authenticate, authorise('admin'), userCtrl.remove);

// Router-level middleware — applies to all routes in THIS router only
router.use((req, res, next) => {
  console.log(`User route accessed: ${req.method} ${req.path}`);
  next();
});

module.exports = router;
```

```
// app.js — mount all routers with base paths
const userRouter    = require('./routes/users');
const productRouter = require('./routes/products');
const orderRouter   = require('./routes/orders');

app.use('/api/v1/users',    userRouter);
app.use('/api/v1/products', productRouter);
app.use('/api/v1/orders',   orderRouter);

// GET /api/v1/users/42 → routes/users.js → router.get('/:id', ...)
```

4. What Is Middleware?

Middleware functions are the building blocks of an Express application. A middleware is any function with the signature (req, res, next) that sits between the incoming request and the final route handler. It can read and modify req/res, end the request-response cycle, or pass control to the next middleware by calling next().

```
// Middleware anatomy
function myMiddleware(req, res, next) {
  // 1. Execute any code
  console.log(`${req.method} ${req.url} at ${new Date().toISOString()}`);

  // 2. Optionally modify req or res
  req.requestTime = Date.now();

  // 3. Either end the cycle...
  // res.status(403).json({ error: 'Forbidden' }); return;

  // ...OR pass to the next middleware/route
  next();
}

// Middleware execution order:
// Request → MW1 → MW2 → Route Handler → MW3 (if next called) → Response

// If any middleware does NOT call next() and does NOT send a response,
// the request hangs forever — a common bug to watch for
```

> *Middleware order matters critically in Express. app.use(cors()) must come before routes so all responses include CORS headers. Error handling middleware (4 arguments) must come last. Authentication middleware must come before protected routes.*

5. Building Custom Middleware

5.1 Request Logger

```
// middleware/logger.js
const logger = (req, res, next) => {
  const start = Date.now();

  // Hook into the response finish event to log after response is sent
  res.on('finish', () => {
    const duration = Date.now() - start;
    const colour   = res.statusCode < 400 ? '\x1b[32m' : '\x1b[31m';
    console.log(
      `${colour}${req.method}\x1b[0m ${req.originalUrl} ` +
      `${res.statusCode} ${duration}ms`
    );
  });

  next();
};

module.exports = logger;

// Usage: app.use(logger);
// Output: GET /api/products 200 12ms
```

5.2 JWT Authentication Middleware

```
// middleware/auth.js
const jwt = require('jsonwebtoken');  // npm install jsonwebtoken

const authenticate = (req, res, next) => {
  const authHeader = req.headers['authorization'];

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'No token provided' });
  }

  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;  // attach user payload to request
    next();
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({ error: 'Token expired' });
    }
    return res.status(401).json({ error: 'Invalid token' });
  }
};

// Role-based authorisation — factory function
const authorise = (...roles) => (req, res, next) => {
```

```
  if (!req.user) {
    return res.status(401).json({ error: 'Not authenticated' });
  }
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ error: 'Insufficient permissions' });
  }
  next();
};

module.exports = { authenticate, authorise };

// Usage in routes:
// router.delete('/:id', authenticate, authorise('admin'), deleteHandler);
```

5.3 Request Validation Middleware

```
// middleware/validate.js — using express-validator
// npm install express-validator
const { body, param, validationResult } = require('express-validator');

// Reusable validation rules for creating a product
const validateProduct = [
  body('name')
    .trim()
    .notEmpty().withMessage('Name is required')
    .isLength({ max: 100 }).withMessage('Name max 100 chars'),

  body('price')
    .notEmpty().withMessage('Price is required')
    .isFloat({ min: 0 }).withMessage('Price must be a positive number'),

  body('category')
    .optional()
    .isIn(['electronics', 'clothing', 'food'])
    .withMessage('Invalid category'),

  // Middleware that checks result and returns 422 if invalid
  (req, res, next) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(422).json({
        error:   'Validation failed',
        details: errors.array().map(e => ({ field: e.path, message: e.msg }))
      });
    }
    next();
  }
];

module.exports = { validateProduct };

// Usage in route:
router.post('/', authenticate, validateProduct, productCtrl.create);
```

5.4 Rate Limiting Middleware

```
// npm install express-rate-limit
const rateLimit = require('express-rate-limit');

// General API rate limit — 100 requests per 15 minutes per IP
const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,   // 15 minutes
  max:      100,
  message:  { error: 'Too many requests, please try again later' },
  standardHeaders: true,       // Return rate limit info in headers
  legacyHeaders:   false,
});

// Stricter limit for auth endpoints — 10 login attempts per 15 min
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max:      10,
  message:  { error: 'Too many login attempts, try again in 15 minutes' },
});

// Apply globally
app.use('/api/', apiLimiter);

// Apply to specific routes
app.use('/api/auth/login',    authLimiter);
app.use('/api/auth/register', authLimiter);
```

6. Third-Party Middleware Essentials

```
const express      = require('express');
const cors         = require('cors');         // npm install cors
const helmet       = require('helmet');       // npm install helmet
const morgan       = require('morgan');       // npm install morgan
const compression  = require('compression'); // npm install compression
const cookieParser = require('cookie-parser');// npm install cookie-parser

const app = express();

// Security — sets various HTTP headers (XSS, CSP, HSTS, etc.)
app.use(helmet());

// CORS — allow cross-origin requests
app.use(cors({ origin: process.env.CLIENT_URL, credentials: true }));

// HTTP request logger
app.use(morgan('dev'));         // 'dev', 'combined', 'tiny'

// Gzip compression for responses
app.use(compression());

// Body parsers
```

```
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true }));

// Cookie parser
app.use(cookieParser(process.env.COOKIE_SECRET));
```

Interview Questions & Answers

Q1: What is middleware in Express and how does it work?

- Middleware is a function with signature (req, res, next) that has access to the request, response, and the next middleware in the chain
- It can execute any code, modify req/res objects, end the request-response cycle, or call next() to pass control to the next function
- Express processes middleware in the order it is registered — this is why order matters: CORS before routes, authentication before protected routes, error handlers last
- If a middleware does not call next() AND does not send a response, the request will hang indefinitely — a common source of bugs

Q2: What is the difference between application-level and router-level middleware?

- Application-level middleware is registered with app.use() or app.get() — it applies to the entire application or all routes matching the given path
- Router-level middleware is registered with router.use() — it applies only to routes defined on that specific Router instance
- Router-level middleware is useful for grouping auth or logging middleware that should only apply to a specific feature area (e.g., all admin routes)
- The behavior is identical — the only difference is scope. Both receive (req, res, next) and use next() in the same way

Q3: What are the four types of middleware in Express?

- Application-level — bound to app with app.use() or app.METHOD(); runs for matching requests across the entire app
- Router-level — bound to a router instance with router.use() or router.METHOD(); scoped to that router
- Error-handling — has FOUR parameters (err, req, res, next); only triggered when next(err) is called or an error is thrown in async code
- Built-in and third-party — express.json(), express.static(), cors(), helmet(), morgan() are all middleware functions applied with app.use()

Q4: How does error-handling middleware differ from regular middleware?

- Error-handling middleware has four parameters: (err, req, res, next) — Express identifies it by the function's arity (4 arguments)
- It is triggered by calling next(err) with an error argument, or by an uncaught error thrown inside a synchronous route handler
- It must be registered LAST — after all routes and other middleware — so it acts as a catch-all for any errors that were not handled earlier
- You can have multiple error-handling middleware functions — e.g., one for logging and one for sending the response — but only the last one should send a response

Q5: What is the purpose of the next() function and next(err)?

- next() — passes control to the next middleware or route handler in the chain. Without calling next() (or sending a response), the chain stops and the request hangs
- next(err) — skips all remaining regular middleware and jumps directly to the nearest error-handling middleware (one with 4 parameters)
- next('route') — skips remaining handlers in the current route and passes to the next matching route — used in router.param() handlers
- Always call return next() or return res.json() to prevent code from continuing to execute after passing control — a missing return causes double response errors

Q6: How do you handle async errors in Express middleware?

- Express 4 does not automatically catch errors thrown from async functions. If an async route throws, the error is silently swallowed unless you catch it
- Wrap async handlers: router.get('/:id', async (req, res, next) => { try { ... } catch(err) { next(err); } })
- Or use a wrapper: const asyncHandler = fn => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next)
- Express 5 (currently in beta/release candidate) automatically catches async errors and forwards them to next() — removing the need for try/catch in every async handler

Q7: What is the purpose of the helmet middleware?

- Helmet sets various security-related HTTP response headers that protect against common web vulnerabilities
- It includes protections for: XSS (x-xss-protection), clickjacking (X-Frame-Options: DENY), MIME sniffing (X-Content-Type-Options: nosniff), and HTTPS enforcement (Strict-Transport-Security)
- app.use(helmet()) applies a sensible set of defaults with one line. Individual sub-middleware can be configured: helmet.contentSecurityPolicy({ directives: {...} })
- Helmet is a collection of 15+ individual middleware functions — using helmet() enables all of them with secure defaults. It should always be the first middleware registered

Q8: What is the difference between app.use('/api', router) and app.use(router)?

- app.use('/api', router) mounts the router at the /api path prefix — all routes defined in the router are relative to /api. router.get('/users') becomes /api/users
- app.use(router) mounts the router without a prefix — routes are relative to the root. router.get('/users') becomes /users
- Mounting with a prefix is the standard practice for versioned APIs: app.use('/api/v1', v1Router) and app.use('/api/v2', v2Router)
- req.url inside the router is relative to the mount point — it shows /users, not /api/v1/users. Use req.originalUrl to get the full path

Q9: How do you implement request validation in Express?

- The express-validator library provides chainable validators: body('email').isEmail(), param('id').isInt(), query('page').isInt({ min: 1 })
- Call validationResult(req) in the final validation middleware to collect all errors — return 422 Unprocessable Entity if errors exist
- Validation middleware is applied as an array before the route handler: router.post('/', [validateUser, handleValidation], userCtrl.create)
- Always sanitise inputs too: body('name').trim().escape() strips whitespace and HTML-encodes special characters to prevent XSS

Q10: What is CORS and how do you configure it in Express?

- CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks JavaScript from making requests to a different origin (domain, port, or protocol) unless the server explicitly permits it
- The cors npm package handles this: app.use(cors()) allows all origins. For production, always restrict: app.use(cors({ origin: 'https://myapp.com' }))
- For credentials (cookies, Authorization headers): set credentials: true and specify the exact origin — wildcards (*) cannot be used with credentials: true
- Preflight requests are automatic OPTIONS requests the browser sends before cross-origin requests with custom headers. cors() handles these automatically by responding to OPTIONS

CHAPTER 4

Database Integration

Connect Node.js to MongoDB with Mongoose and PostgreSQL with pg — queries, models, and best practices

1. Choosing a Database for Node.js

Node.js works with any database that has a JavaScript driver. The two most common pairings are MongoDB (NoSQL document store, paired with Mongoose ODM) and PostgreSQL (relational SQL database, paired with the pg driver or Knex.js/Prisma ORM). Your choice depends on your data shape, query complexity, and consistency requirements.

```
// When to use MongoDB (Mongoose):
// ✓ Flexible, evolving schema — fields vary per document
// ✓ Hierarchical/nested data that maps naturally to JSON
// ✓ Horizontal scaling via sharding is a priority
// ✓ Rapid prototyping — schema changes are cheap

// When to use PostgreSQL (pg/Prisma):
// ✓ Structured data with clear relationships (users, orders, products)
// ✓ Complex queries with JOINs, aggregations, window functions
// ✓ ACID transactions are critical (finance, inventory)
// ✓ Strong data integrity constraints (foreign keys, unique)
```

2. MongoDB with Mongoose

Mongoose is an ODM (Object Document Mapper) for MongoDB. It provides schema definition, validation, middleware hooks, and a rich query API on top of the native MongoDB driver — making it the standard way to use MongoDB in Node.js applications.

2.1 Connecting to MongoDB

```
// npm install mongoose
// config/database.js
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI, {
      // These options are defaults in Mongoose 6+ but good to be explicit
    });
    console.log(`MongoDB connected: ${conn.connection.host}`);
```

```
  } catch (err) {
    console.error('MongoDB connection error:', err.message);
    process.exit(1);   // exit process with failure
  }
};

// Graceful disconnect on app termination
process.on('SIGINT', async () => {
  await mongoose.connection.close();
  console.log('MongoDB connection closed (app termination)');
  process.exit(0);
});

module.exports = connectDB;

// server.js — call before starting server
const connectDB = require('./config/database');
connectDB().then(() => {
  app.listen(PORT, () => console.log(`Server on port ${PORT}`));
});
```

2.2 Defining Mongoose Schemas and Models

```
// models/User.js
const mongoose = require('mongoose');
const bcrypt   = require('bcryptjs');  // npm install bcryptjs

const userSchema = new mongoose.Schema({
  name: {
    type:     String,
    required: [true, 'Name is required'],
    trim:     true,
    maxlength: [50, 'Name cannot exceed 50 characters']
  },
  email: {
    type:     String,
    required: [true, 'Email is required'],
    unique:   true,
    lowercase: true,
    match:    [/^\S+@\S+\.\S+$/, 'Please enter a valid email']
  },
  password: {
    type:     String,
    required: true,
    minlength: 8,
    select:   false   // never return password in queries by default
  },
  role: {
    type:    String,
    enum:    ['user', 'admin', 'editor'],
    default: 'user'
  },
  isActive:  { type: Boolean, default: true },
  lastLogin: { type: Date },
  profile: {
```

```
    bio:    String,
    avatar: String,
  }
}, {
  timestamps: true   // adds createdAt and updatedAt automatically
});

// Index for performance
userSchema.index({ email: 1 });
userSchema.index({ role: 1, isActive: 1 });

// Pre-save hook — hash password before saving
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

// Instance method
userSchema.methods.comparePassword = async function(candidate) {
  return bcrypt.compare(candidate, this.password);
};

// Static method
userSchema.statics.findByEmail = function(email) {
  return this.findOne({ email: email.toLowerCase() }).select('+password');
};

// Virtual — not stored in DB, computed on access
userSchema.virtual('fullProfile').get(function() {
  return `${this.name} <${this.email}>`;
});

module.exports = mongoose.model('User', userSchema);
```

2.3 CRUD Operations with Mongoose

```
const User    = require('./models/User');
const Product = require('./models/Product');

// ── CREATE ──────────────────────────────────────────────────────
const user = await User.create({ name: 'Gowtham', email: 'g@test.com', password:
'pass123' });
// OR: const user = new User({...}); await user.save();

// ── READ ────────────────────────────────────────────────────────
const all   = await User.find();                          // all users
const one   = await User.findById('64abc123...');         // by _id
const byEmail = await User.findOne({ email: 'g@test.com' });

// Filter, select, sort, paginate
const users = await User
  .find({ role: 'user', isActive: true })  // filter
  .select('name email role createdAt')      // fields to return
```

```
  .sort({ createdAt: -1 })                // newest first
  .skip(20)                               // for pagination
  .limit(10);                             // page size 10

// Count
const count = await User.countDocuments({ role: 'user' });

// ── UPDATE ──────────────────────────────────────────────────
// findByIdAndUpdate — returns the UPDATED document
const updated = await User.findByIdAndUpdate(
  id,
  { $set: { name: 'New Name', lastLogin: new Date() } },
  { new: true, runValidators: true }   // new: return updated doc
);

// Common MongoDB update operators:
// $set    — set field value
// $unset  — remove a field
// $inc    — increment a number
// $push   — add to array
// $pull   — remove from array
await Product.findByIdAndUpdate(id, { $inc: { stock: -1 }, $push: { tags: 'sale' }
});

// ── DELETE ──────────────────────────────────────────────────
await User.findByIdAndDelete(id);
await User.deleteMany({ isActive: false });  // bulk delete
```

2.4 Populate — Joining Documents

```
// models/Post.js — references User
const postSchema = new mongoose.Schema({
  title:   { type: String, required: true },
  content: String,
  author:  { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  tags:    [String],
  likes:   [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }]
}, { timestamps: true });

// Query with populate — like a JOIN
const posts = await Post
  .find({ tags: 'node.js' })
  .populate('author', 'name email avatar')    // only these fields from User
  .populate('likes', 'name')                  // populate the likes array
  .sort({ createdAt: -1 })
  .limit(10);

// posts[0].author.name === 'Gowtham'

// Nested populate
const order = await Order
  .findById(id)
  .populate({ path: 'items.product', select: 'name price images' })
  .populate('customer', 'name email');
```

3. PostgreSQL with the pg Driver

For relational data, the pg (node-postgres) library provides a direct, lightweight interface to PostgreSQL. For larger projects, Prisma ORM offers type-safe queries and automatic migrations on top of pg.

3.1 Connection Pool Setup

```
// npm install pg
// config/db.js
const { Pool } = require('pg');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  // OR individual settings:
  // host: process.env.DB_HOST,
  // port: 5432,
  // database: process.env.DB_NAME,
  // user: process.env.DB_USER,
  // password: process.env.DB_PASS,
  max:            10,   // max pool connections
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
  ssl: process.env.NODE_ENV === 'production'
    ? { rejectUnauthorized: false }
    : false
});

// Test connection on startup
pool.query('SELECT NOW()', (err, res) => {
  if (err) { console.error('DB connection error', err); process.exit(1); }
  console.log('PostgreSQL connected:', res.rows[0].now);
});

module.exports = { query: (text, params) => pool.query(text, params), pool };
```

3.2 Parameterised Queries and CRUD

```
const db = require('./config/db');

// ── CREATE ──────────────────────────────────────────────────────
// ALWAYS use parameterised queries — prevents SQL injection
const createUser = async (name, email, hashedPassword) => {
  const result = await db.query(
    `INSERT INTO users (name, email, password_hash, created_at)
     VALUES ($1, $2, $3, NOW())
     RETURNING id, name, email, created_at`,
    [name, email, hashedPassword]
  );
  return result.rows[0];
```

```
};

// ── READ ──────────────────────────────────────────────────────────
const getUsers = async ({ page = 1, limit = 10, role } = {}) => {
  const offset = (page - 1) * limit;
  const params = [limit, offset];
  let query = 'SELECT id, name, email, role, created_at FROM users';

  if (role) {
    query += ' WHERE role = $3';
    params.push(role);
  }

  query += ' ORDER BY created_at DESC LIMIT $1 OFFSET $2';
  const { rows } = await db.query(query, params);
  return rows;
};

const getUserById = async (id) => {
  const { rows } = await db.query(
    'SELECT id, name, email, role FROM users WHERE id = $1',
    [id]
  );
  return rows[0] || null;
};

// ── UPDATE ────────────────────────────────────────────────────────
const updateUser = async (id, { name, email }) => {
  const { rows } = await db.query(
    `UPDATE users SET name=$1, email=$2, updated_at=NOW()
     WHERE id=$3 RETURNING id, name, email`,
    [name, email, id]
  );
  return rows[0];
};

// ── DELETE ────────────────────────────────────────────────────────
const deleteUser = async (id) => {
  const { rowCount } = await db.query('DELETE FROM users WHERE id=$1', [id]);
  return rowCount > 0;
};
```

3.3 Transactions with pg

```
// Transactions ensure all operations succeed or all are rolled back
const transferFunds = async (fromId, toId, amount) => {
  const client = await pool.connect();

  try {
    await client.query('BEGIN');

    // Debit sender
    const debit = await client.query(
```

```
      'UPDATE accounts SET balance = balance - $1 WHERE id = $2 AND balance >= $1
RETURNING balance',
      [amount, fromId]
    );

    if (debit.rowCount === 0) {
      throw new Error('Insufficient funds');
    }

    // Credit receiver
    await client.query(
      'UPDATE accounts SET balance = balance + $1 WHERE id = $2',
      [amount, toId]
    );

    // Record transaction
    await client.query(
      'INSERT INTO transfers (from_id, to_id, amount) VALUES ($1, $2, $3)',
      [fromId, toId, amount]
    );

    await client.query('COMMIT');
    return { success: true };
  } catch (err) {
    await client.query('ROLLBACK');
    throw err;
  } finally {
    client.release();   // ALWAYS release back to pool
  }
};
```

> *Always release pg pool clients in a finally block. If an error is thrown before client.release(), the client is never returned to the pool — eventually exhausting all connections and hanging every subsequent request.*

4. Prisma ORM — Type-Safe Database Access

Prisma is a modern Node.js ORM that auto-generates a type-safe client from your schema. It supports PostgreSQL, MySQL, SQLite, and MongoDB, and produces TypeScript types automatically from your schema file.

```
# npm install prisma @prisma/client
# npx prisma init

// prisma/schema.prisma
generator client {
  provider = 'prisma-client-js'
}

datasource db {
```

```
  provider = 'postgresql'
  url      = env('DATABASE_URL')
}

model User {
  id        Int      @id @default(autoincrement())
  email     String   @unique
  name      String
  role      Role     @default(USER)
  posts     Post[]
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

model Post {
  id        Int      @id @default(autoincrement())
  title     String
  content   String?
  published Boolean  @default(false)
  author    User     @relation(fields: [authorId], references: [id])
  authorId  Int
}

enum Role { USER ADMIN EDITOR }

# Generate the client after schema changes
# npx prisma generate
# npx prisma migrate dev --name init
```

```
// Using Prisma Client
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

// CREATE
const user = await prisma.user.create({
  data: { email: 'g@test.com', name: 'Gowtham', role: 'ADMIN' }
});

// READ with relations
const users = await prisma.user.findMany({
  where:   { role: 'USER', posts: { some: { published: true } } },
  select:  { id: true, name: true, email: true, _count: { select: { posts: true }
} },
  orderBy: { createdAt: 'desc' },
  skip:    0,
  take:    10,
});

// Include related data
const userWithPosts = await prisma.user.findUnique({
  where:   { id: 1 },
  include: { posts: { where: { published: true }, orderBy: { createdAt: 'desc' } }
}
});
```

```
// UPDATE
const updated = await prisma.user.update({
  where: { id: 1 },
  data:  { name: 'Updated Name' }
});

// DELETE
await prisma.user.delete({ where: { id: 1 } });

// Transaction
await prisma.$transaction([
  prisma.account.update({ where: { id: 1 }, data: { balance: { decrement: 100 } }
}),
  prisma.account.update({ where: { id: 2 }, data: { balance: { increment: 100 } }
}),
]);
```

Interview Questions & Answers

Q1: What is the difference between MongoDB and PostgreSQL for Node.js?

- MongoDB is a NoSQL document store — data is stored as flexible JSON-like BSON documents. No fixed schema, easy to scale horizontally, great for nested/hierarchical data
- PostgreSQL is a relational SQL database — data is stored in tables with rows and columns, enforced schemas, foreign keys, and full ACID transaction support
- MongoDB excels for rapidly evolving schemas, content management systems, and real-time applications. PostgreSQL excels for financial data, complex reporting, and anything requiring strict relational integrity
- Both have excellent Node.js drivers (mongoose for MongoDB, pg/Prisma for PostgreSQL) and both support indexing, aggregation, and high throughput

Q2: What is Mongoose and what does it add over the native MongoDB driver?

- Mongoose is an ODM (Object Document Mapper) that adds schema definition, validation, middleware hooks, and a higher-level query API on top of the native mongodb driver
- Schema definition lets you specify field types, required fields, defaults, enum values, and custom validators — enforcing structure on an otherwise schemaless database
- Middleware hooks (pre/post save, find, remove) let you run logic automatically — hash passwords before save, update timestamps after find, cascade deletes
- Populate() provides JOIN-like functionality to replace ObjectID references with the actual documents from other collections

Q3: Why should you always use parameterised queries in SQL?

- Parameterised queries (db.query('SELECT * FROM users WHERE id=$1', [id])) pass user input as separate parameters — the database driver treats them as data, never as SQL code
- Without parameterisation, SQL injection is possible: if id = '1; DROP TABLE users--', a naive string-interpolated query would execute the DROP TABLE command
- Even if your current inputs are safe, parameterisation is non-negotiable as a defence-in-depth practice — no input should ever be directly interpolated into a SQL string
- Parameterised queries are also faster on repeated executions — PostgreSQL can cache the query execution plan after the first run

Q4: What is a connection pool and why is it important?

- A connection pool maintains a set of pre-established database connections that can be reused across requests — instead of opening a new connection for every query (which takes 20-100ms for TCP handshake + DB auth)
- Without pooling, a server under load would either exhaust DB connections or add significant latency on every request
- The pg Pool class manages a pool of connections (default max 10). mongoose.connect() also maintains a pool internally
- Always configure max pool size based on your database server's max_connections limit and expected concurrency — too many pools across multiple app instances can exhaust the DB

Q5: What is a database transaction and when do you need one?

- A transaction is a sequence of database operations that are executed as a single atomic unit — either all succeed (COMMIT) or all are rolled back (ROLLBACK) leaving the database unchanged
- ACID properties: Atomicity (all or nothing), Consistency (data stays valid), Isolation (concurrent transactions don't interfere), Durability (committed data persists)
- Use transactions for: bank transfers (debit + credit must both succeed), order creation (create order + reduce stock + charge payment), or any multi-step operation where partial completion would leave corrupt state
- In pg: get a client from the pool, call BEGIN, execute queries, call COMMIT on success or ROLLBACK on error, always release the client in finally

Q6: What are Mongoose middleware hooks and what are they used for?

- Mongoose middleware (also called pre/post hooks) are functions that run before or after specific operations: save, find, findOne, update, delete, validate
- pre('save') is the most common — used to hash passwords, generate slugs, or set computed fields before a document is persisted
- post('save') runs after a document is saved — used for sending welcome emails, creating audit logs, or invalidating caches
- Hooks are defined on the schema before creating the model: schema.pre('save', async function(next) { ... next(); }) — use regular functions (not arrow functions) to access this (the document)

Q7: What is Prisma and what advantages does it offer over raw SQL or Mongoose?

- Prisma is a type-safe ORM — it generates TypeScript types from your schema file, so your IDE auto-completes fields and TypeScript catches wrong field names at compile time
- Prisma Migrate generates and tracks SQL migration files automatically from schema changes — no manual SQL migration files to write
- The Prisma Client API is explicit and readable: prisma.user.findMany({ where: { role: 'ADMIN' }, include: { posts: true } }) clearly expresses intent
- Prisma works with PostgreSQL, MySQL, SQLite, SQL Server, and MongoDB — one schema syntax for multiple databases

Q8: How do you implement pagination in a database query?

- Offset pagination: LIMIT 10 OFFSET 20 (pg) or .skip(20).limit(10) (Mongoose) — simple but slow on large datasets because the DB must scan all preceding rows
- Cursor pagination: WHERE id > lastSeenId LIMIT 10 — more efficient for large datasets; the DB uses an index seek instead of a full scan. Requires a stable sort field
- Always return total count alongside the page: countDocuments({filter}) + find({filter}).skip().limit() — or a single aggregation pipeline with $facet in MongoDB
- For the frontend: return { data: [...], total: 150, page: 2, limit: 10, totalPages: 15, hasNextPage: true }

Q9: What is the N+1 query problem and how do you solve it?

- N+1 happens when you fetch N items and then execute a separate query for each item's related data — resulting in 1 + N database queries instead of 1
- Example: fetching 10 posts (1 query) then fetching each post's author separately (10 queries) = 11 total queries when 2 would suffice
- Solution in Mongoose: use .populate() to JOIN in a single additional query. Solution in SQL: use JOIN in the original query instead of a loop
- In Prisma: use include — prisma.post.findMany({ include: { author: true } }) fetches posts and authors in 2 optimised queries

Q10: What are MongoDB indexes and why are they important?

- An index is a data structure (typically B-tree) that stores a sorted copy of one or more fields, allowing MongoDB to locate documents without scanning every document in the collection
- Without an index on the queried field, MongoDB performs a full collection scan — O(n) — which is very slow on millions of documents
- Create indexes in Mongoose schema: userSchema.index({ email: 1 }) for ascending, userSchema.index({ createdAt: -1 }) for descending, userSchema.index({ email: 1, role: 1 }) for compound

- The email field should always be indexed if you query by it (findOne({ email })). Over-indexing is also a problem — indexes slow down writes and consume RAM, so only index fields you actually query or sort by

CHAPTER 5

Async Programming

Master callbacks, Promises, async/await, and concurrent patterns in Node.js

1. Why Async Programming Matters in Node.js

Node.js is single-threaded. Every time your code waits for I/O (database queries, file reads, HTTP requests), it must do so without blocking the main thread — otherwise no other request can be processed. JavaScript provides three patterns for handling async operations: callbacks, Promises, and async/await. Each is an evolution of the previous.

> *Rule of thumb: always use async/await for new code. Use Promises when dealing with multiple concurrent operations (Promise.all). Understand callbacks because Node.js core APIs and many older libraries still use them.*

2. Callbacks — The Original Pattern

A callback is a function you pass to another function that will be called when the operation completes. Node.js uses the error-first callback convention: the first argument is always an error (null if none), and the second is the result.

```
const fs = require('fs');

// Error-first callback pattern
fs.readFile('./data.json', 'utf8', (err, data) => {
  if (err) {
    console.error('Failed to read file:', err.message);
    return;   // stop execution
  }
  const parsed = JSON.parse(data);
  console.log('Data:', parsed);
});

// ── Callback Hell — the problem with callbacks ──────────────────
// Reading file → parsing → querying DB → processing → writing result
fs.readFile('config.json', 'utf8', (err, config) => {
  if (err) return handleError(err);
  db.findUser(JSON.parse(config).userId, (err, user) => {
    if (err) return handleError(err);
    db.findOrders(user.id, (err, orders) => {
      if (err) return handleError(err);
      processOrders(orders, (err, result) => {
```

```
        if (err) return handleError(err);
        fs.writeFile('result.json', JSON.stringify(result), (err) => {
          if (err) return handleError(err);
          console.log('Done!');  // Pyramid of doom!
        });
      });
    });
  });
});
```

3. Promises — Solving Callback Hell

A Promise is an object representing the eventual completion or failure of an asynchronous operation. It has three states: pending, fulfilled (resolved with a value), or rejected (failed with a reason). Promises can be chained with .then() and errors are caught with .catch().

```
// Creating a Promise
const readFileAsync = (path) => {
  return new Promise((resolve, reject) => {
    fs.readFile(path, 'utf8', (err, data) => {
      if (err) reject(err);     // failure
      else     resolve(data);   // success
    });
  });
};

// Consuming Promises — .then() chaining
readFileAsync('data.json')
  .then(data => JSON.parse(data))                      // transform
  .then(parsed => db.saveUser(parsed))                 // async chain
  .then(user => console.log('Saved:', user.id))
  .catch(err => console.error('Error:', err.message))  // single error handler
  .finally(() => console.log('Done — runs always'));   // cleanup

// Promise.resolve() / Promise.reject() — wrap values in promises
Promise.resolve(42).then(v => console.log(v));         // 42
Promise.reject(new Error('bad')).catch(e => console.error(e.message));

// util.promisify — convert callback-style functions to Promises
const { promisify } = require('util');
const readFile = promisify(fs.readFile);
readFile('./data.json', 'utf8')
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

4. async/await — Clean Async Code

async/await is syntactic sugar over Promises. An async function always returns a Promise. The await keyword pauses execution inside the async function until the Promise resolves — without

blocking the event loop. This makes async code look and behave like synchronous code, dramatically improving readability.

```
// async/await — same operations as callback hell, but clean
const processUserData = async () => {
  try {
    const config  = JSON.parse(await fs.promises.readFile('config.json', 'utf8'));
    const user    = await db.findUser(config.userId);
    const orders  = await db.findOrders(user.id);
    const result  = await processOrders(orders);
    await fs.promises.writeFile('result.json', JSON.stringify(result));
    console.log('Done!');
  } catch (err) {
    console.error('Error:', err.message);
  }
};

// Express route with async/await
router.get('/users/:id', async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ error: 'User not found' });
    res.json({ success: true, data: user });
  } catch (err) {
    next(err);   // pass to error handler
  }
});

// async IIFE — run async code at top level in CJS
(async () => {
  await connectDB();
  app.listen(PORT, () => console.log('Ready'));
})();

// Top-level await in ES Modules (no IIFE needed)
await connectDB();
app.listen(PORT);
```

5. Concurrent Operations — Promise Combinators

When multiple async operations are independent of each other, running them concurrently is far faster than awaiting each one sequentially. Promise combinators like Promise.all and Promise.allSettled enable this.

```
// ── SEQUENTIAL (slow) — 3 seconds total ──────────────────────────
const user     = await fetchUser(id);     // 1 second
const orders   = await fetchOrders(id);   // 1 second
const settings = await fetchSettings(id); // 1 second

// ── CONCURRENT (fast) — 1 second total ──────────────────────────
const [user, orders, settings] = await Promise.all([
```

```
  fetchUser(id),
  fetchOrders(id),
  fetchSettings(id)
]);
// All three fire simultaneously — total time = slowest individual request

// Promise.all — rejects immediately if ANY promise rejects
try {
  const results = await Promise.all([p1, p2, p3]);
} catch (err) {
  // If p2 rejects, p1 and p3 results are lost
}

// Promise.allSettled — waits for ALL promises regardless of rejection
const results = await Promise.allSettled([p1, p2, p3]);
results.forEach(r => {
  if (r.status === 'fulfilled') console.log('Value:', r.value);
  if (r.status === 'rejected')  console.error('Error:', r.reason);
});

// Promise.race — resolves/rejects with the FIRST promise to settle
// Use for: implementing timeouts
const withTimeout = (promise, ms) => Promise.race([
  promise,
  new Promise((_, reject) =>
    setTimeout(() => reject(new Error(`Timed out after ${ms}ms`)), ms)
  )
]);
const data = await withTimeout(fetchData(), 5000);

// Promise.any — resolves with the FIRST fulfillment (ignores rejections)
// Use for: trying multiple sources and taking the fastest successful one
const fastest = await Promise.any([fetchFromCDN1(), fetchFromCDN2(),
fetchFromOrigin()]);
```

6. Async Patterns in Express

```
// ── asyncHandler wrapper — eliminates try/catch boilerplate ──────
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

// Clean routes without try/catch
router.get('/',     asyncHandler(getAllProducts));
router.get('/:id',  asyncHandler(getProductById));
router.post('/',    asyncHandler(createProduct));

async function getAllProducts(req, res) {
  const { page = 1, limit = 10 } = req.query;
  const products = await Product.find()
    .skip((page - 1) * limit)
    .limit(Number(limit));
  const total = await Product.countDocuments();
  res.json({ success: true, data: products, total, page: Number(page) });
```

```
}

// ── Parallel operations in a route ──────────────────────────────
async function getDashboard(req, res) {
  const userId = req.user.id;

  const [user, orders, notifications, stats] = await Promise.all([
    User.findById(userId).select('-password'),
    Order.find({ userId }).sort({ createdAt: -1 }).limit(5),
    Notification.find({ userId, read: false }),
    Order.aggregate([{ $match: { userId } }, { $group: { _id: null, total: { $sum:
'$amount' } } }])
  ]);

  res.json({ user, orders, unreadCount: notifications.length, totalSpent:
stats[0]?.total ?? 0 });
}

// ── Streaming large datasets (avoid loading into memory) ─────────
async function exportUsers(req, res) {
  res.setHeader('Content-Type', 'application/json');
  res.write('[');
  let first = true;

  const cursor = User.find().cursor();
  for await (const user of cursor) {
    if (!first) res.write(',');
    res.write(JSON.stringify(user));
    first = false;
  }

  res.write(']');
  res.end();
}
```

7. Worker Threads — CPU-Intensive Tasks

Worker threads allow Node.js to run JavaScript in separate threads in parallel, enabling true parallel computation for CPU-intensive tasks without blocking the event loop.

```
// ── Main thread — delegates CPU work to a worker ────────────────
const { Worker } = require('worker_threads');

function runWorker(workerData) {
  return new Promise((resolve, reject) => {
    const worker = new Worker('./workers/hash-worker.js', { workerData });
    worker.on('message', resolve);
    worker.on('error',   reject);
    worker.on('exit', code => {
      if (code !== 0) reject(new Error(`Worker stopped with exit code ${code}`));
    });
  });
```

```
}

router.post('/hash', asyncHandler(async (req, res) => {
  // Offload bcrypt hashing to a worker thread — does not block event loop
  const hash = await runWorker({ password: req.body.password, rounds: 12 });
  res.json({ hash });
}));

// ── workers/hash-worker.js ──────────────────────────────────
const { workerData, parentPort } = require('worker_threads');
const bcrypt = require('bcryptjs');

bcrypt.hash(workerData.password, workerData.rounds)
  .then(hash => parentPort.postMessage(hash));
```

Interview Questions & Answers

Q1: What are the three async patterns in Node.js and how do they relate?

- Callbacks (original) — pass a function to be called when the operation completes. Error-first convention: (err, result). Problem: callback hell when chaining multiple async operations
- Promises — an object representing future success or failure. Chainable with .then() and .catch(), solving callback hell. Introduced in ES6
- async/await (modern) — syntactic sugar over Promises. async functions return Promises; await pauses execution until a Promise resolves. Makes async code look synchronous. Introduced in ES2017
- They are not competing — they work together. async/await is built on Promises; Promises can wrap callbacks. Most modern Node.js code uses async/await with Promise.all for concurrent operations

Q2: What is the difference between Promise.all and Promise.allSettled?

- Promise.all([p1, p2, p3]) waits for ALL promises to fulfill. If ANY promise rejects, it rejects immediately with that error and the results of other promises are discarded
- Promise.allSettled([p1, p2, p3]) waits for ALL promises to settle (fulfill OR reject). Returns an array of result objects with { status: 'fulfilled', value } or { status: 'rejected', reason }
- Use Promise.all when all results are required and a single failure should stop everything (e.g., parallel API calls where all are critical)
- Use Promise.allSettled when you want to process as many results as possible and handle failures individually (e.g., sending emails to multiple users — one failure should not stop the rest)

Q3: What is async/await and what are its common pitfalls?

- async/await makes asynchronous Promise-based code look and behave like synchronous code — improving readability and reducing nesting
- Pitfall 1 — sequential instead of parallel: await a(); await b() runs sequentially. If they are independent, use await Promise.all([a(), b()]) to run concurrently
- Pitfall 2 — unhandled rejections: always wrap await calls in try/catch or use an asyncHandler wrapper in Express. Unhandled rejections crash the process in Node 15+
- Pitfall 3 — await inside forEach: forEach does not wait for async callbacks. Use for...of loop or Promise.all with .map() instead

Q4: What happens if you use await inside a forEach loop?

- Array.forEach() does not await the async callback — it fires all callbacks synchronously and does not wait for any of them to resolve
- This means all iterations start in parallel without waiting for each other, and code after the forEach continues before any iteration completes
- Solution for sequential: use for...of loop: for (const item of items) { await processItem(item); }
- Solution for parallel: use Promise.all with map: await Promise.all(items.map(item => processItem(item)))

Q5: What is Promise.race and when would you use it?

- Promise.race([p1, p2, p3]) resolves or rejects as soon as the FIRST promise settles — it adopts the first outcome, whether fulfilled or rejected
- Most common use case: implementing request timeouts — race the real request against a setTimeout promise that rejects after N milliseconds
- const result = await Promise.race([fetchData(), rejectAfter(5000, 'Timeout')]) — if fetchData takes longer than 5 seconds, the race rejects with 'Timeout'
- Also useful for trying multiple sources and taking the first successful response, though Promise.any is better for this (it ignores rejections)

Q6: What is a Promise chain and how does error propagation work?

- A Promise chain is a series of .then() calls where each handler's return value becomes the input to the next .then()
- If any .then() handler throws an error or returns a rejected Promise, the chain skips all subsequent .then() handlers and jumps to the nearest .catch()
- This means one .catch() at the end of a chain handles errors from any step — unlike callbacks where each step needs its own error check
- .finally() runs regardless of success or failure — ideal for cleanup like closing loading spinners, releasing resources, or hiding loaders

Q7: What is util.promisify and when do you need it?

- util.promisify() converts a function that uses the Node.js error-first callback convention into a function that returns a Promise

- Many Node.js core APIs (fs.readFile, fs.writeFile, dns.lookup) use callbacks — promisify wraps them: const readFile = promisify(fs.readFile)
- Modern Node.js ships promise-based alternatives (fs.promises.readFile, dns.promises.lookup) — prefer these over promisify when available
- For libraries that do not follow the error-first callback convention, wrap them manually: new Promise((resolve, reject) => library.method(args, (result) => resolve(result)))

Q8: What are Worker Threads and when should you use them in Node.js?

- Worker Threads (worker_threads module) run JavaScript code in a separate thread in parallel — enabling true multi-threading in Node.js
- Use them for CPU-intensive operations: bcrypt hashing, image resizing, PDF generation, complex data transformations — anything that would block the event loop for more than a few milliseconds
- Workers communicate with the main thread via postMessage/on('message') — they do not share the same memory by default (use SharedArrayBuffer for shared memory)
- Do not use workers for I/O — Node's async I/O model already handles I/O concurrently without threads. Workers are specifically for CPU-bound work

Q9: What is the asyncHandler pattern and why is it used in Express?

- asyncHandler wraps an async route handler in a function that catches any rejected promise and forwards it to next(err): const asyncHandler = fn => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next)
- Without it, every async route needs try/catch: async (req, res, next) => { try { ... } catch(err) { next(err); } } — boilerplate that obscures the actual logic
- With asyncHandler, routes are clean: router.get('/', asyncHandler(getAllProducts)) and errors automatically reach the global error handler
- The express-async-errors npm package monkey-patches Express to automatically catch async errors without any wrapper — a zero-boilerplate alternative

Q10: How do you handle multiple async operations that depend on each other vs. those that are independent?

- Dependent (sequential): use await in sequence — const user = await getUser(id); const orders = await getOrders(user.id) — the second depends on the first's result
- Independent (concurrent): use Promise.all — const [user, products] = await Promise.all([getUser(id), getProducts()]) — both fire simultaneously, total time = slowest
- Mixed: await the dependencies, then run independent operations concurrently: const user = await getUser(id); const [orders, settings] = await Promise.all([getOrders(user.id), getSettings(user.id)])
- The performance difference is significant: 3 independent 200ms operations take 600ms sequentially but only 200ms with Promise.all

CHAPTER 6

Error Handling

Catch, classify, and respond to errors correctly — building resilient production Node.js APIs

1. Why Error Handling Is Critical

In production, errors are inevitable — network failures, invalid input, database outages, programming bugs. A well-designed error handling strategy ensures your API returns meaningful error responses, never exposes sensitive internals to clients, logs enough information for debugging, and keeps the process running through recoverable errors.

> *A Node.js process that crashes on every unhandled error takes down ALL users, not just the one who triggered the bug. Proper error handling is the difference between a resilient production API and an unreliable one.*

```
// Types of errors in a Node.js application:

// 1. Operational errors — expected, predictable failures
//    Network timeout, DB connection lost, validation failure,
//    resource not found, rate limit exceeded
//    → Handle gracefully, return appropriate HTTP status

// 2. Programmer errors — bugs in the code
//    Undefined is not a function, reading property of null,
//    wrong type passed to a function
//    → These are bugs — fix them, do not try to recover

// 3. Uncaught exceptions — errors that escaped all handlers
//    → Log, alert, restart the process gracefully
```

2. Creating Custom Error Classes

A custom error class carries both an HTTP status code and a human-readable message. This lets the global error handler format a consistent response without needing conditional logic for every error type.

```
// utils/AppError.js
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
```

```
    this.statusCode  = statusCode;
    this.status      = statusCode < 500 ? 'fail' : 'error';
    this.isOperational = true;  // marks it as expected/handled

    // Captures where the error was created (excludes this constructor)
    Error.captureStackTrace(this, this.constructor);
  }
}

// Specialised error subclasses
class NotFoundError extends AppError {
  constructor(resource = 'Resource') {
    super(`${resource} not found`, 404);
  }
}

class ValidationError extends AppError {
  constructor(message, fields = []) {
    super(message, 422);
    this.fields = fields;
  }
}

class UnauthorizedError extends AppError {
  constructor(message = 'Authentication required') {
    super(message, 401);
  }
}

class ForbiddenError extends AppError {
  constructor(message = 'Insufficient permissions') {
    super(message, 403);
  }
}

class ConflictError extends AppError {
  constructor(message) { super(message, 409); }
}

module.exports = { AppError, NotFoundError, ValidationError,
                   UnauthorizedError, ForbiddenError, ConflictError };
```

3. The Global Error Handler

The global error handling middleware is a single place that handles ALL errors — whether thrown directly, passed via next(err), or caught from async code. It runs after all routes and transforms errors into consistent JSON responses.

```
// middleware/errorHandler.js
const { AppError } = require('../utils/AppError');
```

```
// Handle specific Mongoose error types
const handleMongooseCastError = (err) =>
  new AppError(`Invalid ${err.path}: ${err.value}`, 400);

const handleMongooseDuplicateKey = (err) => {
  const field = Object.keys(err.keyValue)[0];
  return new AppError(`${field} already exists`, 409);
};

const handleMongooseValidation = (err) => {
  const messages = Object.values(err.errors).map(e => e.message);
  return new AppError(`Validation failed: ${messages.join('. ')}`, 422);
};

const handleJWTError = () => new AppError('Invalid token. Please log in again.',
401);
const handleJWTExpired = () => new AppError('Token expired. Please log in again.',
401);

// Development response — include full stack trace
const sendDevError = (err, res) => {
  res.status(err.statusCode).json({
    status:  err.status,
    error:   err,
    message: err.message,
    stack:   err.stack
  });
};

// Production response — safe, no internals exposed
const sendProdError = (err, res) => {
  if (err.isOperational) {
    // Operational — safe to show client
    res.status(err.statusCode).json({
      status:  err.status,
      message: err.message,
      ...(err.fields && { fields: err.fields })
    });
  } else {
    // Programming error — do NOT leak details
    console.error('UNEXPECTED ERROR:', err);
    res.status(500).json({
      status:  'error',
      message: 'Something went wrong. Please try again later.'
    });
  }
};

// The global error handler — 4 parameters (err, req, res, next)
const errorHandler = (err, req, res, next) => {
  err.statusCode = err.statusCode || 500;
  err.status     = err.status     || 'error';

  if (process.env.NODE_ENV === 'development') {
    sendDevError(err, res);
  } else {
    let error = { ...err, message: err.message };
```

```
    // Convert known error types to AppError
    if (err.name === 'CastError')            error = handleMongooseCastError(err);
    if (err.code === 11000)                  error =
handleMongooseDuplicateKey(err);
    if (err.name === 'ValidationError')      error =
handleMongooseValidation(err);
    if (err.name === 'JsonWebTokenError')    error = handleJWTError();
    if (err.name === 'TokenExpiredError')    error = handleJWTExpired();

    sendProdError(error, res);
  }
};

module.exports = errorHandler;

// app.js — MUST be registered last, after all routes
app.use(errorHandler);
```

4. Using Custom Errors in Routes

```
const { NotFoundError, ValidationError, ConflictError } =
require('../utils/AppError');
const asyncHandler = require('../utils/asyncHandler');

// Route handler — throw errors, global handler catches them
router.get('/:id', asyncHandler(async (req, res) => {
  const { id } = req.params;

  if (!mongoose.Types.ObjectId.isValid(id)) {
    throw new ValidationError('Invalid product ID format');
  }

  const product = await Product.findById(id);
  if (!product) throw new NotFoundError('Product');

  res.json({ success: true, data: product });
}));

router.post('/', asyncHandler(async (req, res) => {
  const { name, email } = req.body;

  // Check for duplicate
  const exists = await User.findOne({ email });
  if (exists) throw new ConflictError('Email already registered');

  const user = await User.create(req.body);
  res.status(201).json({ success: true, data: user });
}));

// 404 route — catches all unmatched routes
app.all('*', (req, res, next) => {
```

```
  next(new NotFoundError(`Route ${req.method} ${req.originalUrl}`));
});
```

5. Handling Uncaught Exceptions and Rejections

Some errors escape all try/catch blocks and route handlers. Node.js provides two global handlers for these. The correct response is to log the error, close the server gracefully, and restart the process via a process manager like PM2.

```
// server.js — place BEFORE app.listen()

// ── Uncaught Exceptions — synchronous errors with no try/catch ─────
process.on('uncaughtException', (err) => {
  console.error('UNCAUGHT EXCEPTION! Shutting down...');
  console.error(err.name, err.message, err.stack);

  // Exit immediately — process is in undefined state
  process.exit(1);
});

// ── Unhandled Promise Rejections ──────────────────────────────────
process.on('unhandledRejection', (err) => {
  console.error('UNHANDLED REJECTION! Shutting down...');
  console.error(err.name, err.message);

  // Graceful shutdown — let ongoing requests finish (up to 30s)
  server.close(() => {
    process.exit(1);
  });

  // Force exit if server.close takes too long
  setTimeout(() => process.exit(1), 30000).unref();
});

// ── SIGTERM — graceful shutdown on deploy/container stop ──────────
process.on('SIGTERM', () => {
  console.log('SIGTERM received. Closing HTTP server...');
  server.close(() => {
    console.log('HTTP server closed. Process will exit.');
    mongoose.connection.close();
  });
});

const server = app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

6. Logging with Winston

console.log is fine for development but inadequate for production. Winston is the most popular Node.js logging library — it supports structured logging, multiple transports (file, console, external services), and different log levels.

```
// npm install winston
// utils/logger.js
const winston = require('winston');
const path    = require('path');

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp({ format: 'YYYY-MM-DD HH:mm:ss' }),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  defaultMeta: { service: 'my-api', env: process.env.NODE_ENV },
  transports: [
    // Console transport — coloured in development
    new winston.transports.Console({
      format: process.env.NODE_ENV === 'development'
        ? winston.format.combine(
            winston.format.colorize(),
            winston.format.simple()
          )
        : winston.format.json()
    }),

    // File transports — persistent logs
    new winston.transports.File({
      filename: path.join('logs', 'error.log'),
      level:    'error'
    }),
    new winston.transports.File({
      filename: path.join('logs', 'combined.log')
    })
  ]
});

module.exports = logger;

// Usage throughout the app
const logger = require('./utils/logger');

logger.info('Server started', { port: 3000 });
logger.warn('Rate limit approaching', { ip: req.ip, count: 95 });
logger.error('Database query failed', { error: err.message, query: sql });

// In global error handler
const errorHandler = (err, req, res, next) => {
  logger.error('Request error', {
    message:    err.message,
    statusCode: err.statusCode,
    path:       req.path,
    method:     req.method,
    ip:         req.ip,
```

```
    stack:      err.stack
  });
  // ... send response
};
```

7. Complete Production-Ready App Structure

```
my-api/
├── src/
│   ├── config/
│   │   ├── database.js      ← DB connection
│   │   └── env.js           ← env var validation
│   ├── controllers/
│   │   ├── user.controller.js
│   │   └── product.controller.js
│   ├── middleware/
│   │   ├── auth.js          ← JWT authenticate + authorise
│   │   ├── errorHandler.js  ← global error handler
│   │   ├── logger.js        ← request logger
│   │   ├── notFound.js      ← 404 handler
│   │   └── validate.js      ← request validation
│   ├── models/
│   │   ├── User.js
│   │   └── Product.js
│   ├── routes/
│   │   ├── auth.routes.js
│   │   ├── user.routes.js
│   │   └── product.routes.js
│   ├── services/            ← business logic
│   │   └── email.service.js
│   └── utils/
│       ├── AppError.js
│       ├── asyncHandler.js
│       └── logger.js
├── app.js                   ← Express setup (no listen)
├── server.js                ← process handlers + app.listen
├── .env
├── .gitignore
└── package.json
```

Interview Questions & Answers

Q1: What is the difference between operational errors and programmer errors?

- Operational errors are expected, predictable failures: network timeouts, database connection lost, invalid user input, resource not found, rate limit exceeded. They should be handled gracefully and return appropriate HTTP status codes

- Programmer errors are bugs: TypeError (cannot read property of undefined), wrong argument types, accessing undefined variables. These are mistakes in the code — they should not be caught and 'handled'; they should be fixed
- The distinction matters for error handling strategy: operational errors should be caught and turned into user-facing responses. Programmer errors should crash the process (so the process manager restarts with clean state) and be fixed in code
- isOperational: true on custom error classes lets the global error handler distinguish between these two types

Q2: How does the global error handling middleware work in Express?

- Error-handling middleware has four parameters: (err, req, res, next) — Express recognises it as an error handler by the function arity
- It is triggered by calling next(err) from any route or middleware, by throwing an error in a synchronous handler, or (with asyncHandler) by a rejected Promise
- It must be registered LAST — after all routes and regular middleware — so errors from any route bubble up to it
- Best practice: have two modes — development (returns full error + stack trace) and production (returns safe generic messages for programmer errors, detailed messages only for operational errors)

Q3: What is the purpose of creating custom error classes?

- Custom error classes (extending Error) let you attach metadata to errors — statusCode, isOperational, field names — without passing separate parameters
- They allow the global error handler to format consistent responses: every error that reaches it already has statusCode and message set
- Throwing semantically named errors makes route code readable: throw new NotFoundError('Product') is more expressive than res.status(404).json({ error: 'Product not found' }) scattered across every route
- Error.captureStackTrace(this, this.constructor) cleans up the stack trace so it starts at the throw site, not inside the constructor

Q4: What are uncaughtException and unhandledRejection and how should you handle them?

- uncaughtException fires when a synchronous error is thrown and not caught by any try/catch — the process is in an unknown, potentially corrupt state
- unhandledRejection fires when a Promise rejects without a .catch() or try/catch — common in async code where await was forgotten
- The correct response for both: log the error with full details, close the server gracefully with server.close(), then call process.exit(1) to restart via PM2 or Docker
- Never just log and continue after uncaughtException — the process state is undefined and continuing risks data corruption or silent incorrect behaviour

Q5: What is SIGTERM and why should your Node.js app handle it?

- SIGTERM is a signal sent to a process asking it to terminate gracefully — it is sent by process managers (PM2), container orchestrators (Kubernetes, Docker), and cloud platforms during deployment or scaling
- Without a SIGTERM handler, the process exits immediately — dropping in-flight requests, leaving DB transactions incomplete, and potentially corrupting caches
- With a handler: process.on('SIGTERM', () => server.close(() => { db.disconnect(); process.exit(0); })) — lets ongoing requests finish before shutting down
- Kubernetes sends SIGTERM then waits 30 seconds (terminationGracePeriodSeconds) before sending SIGKILL. Your graceful shutdown must complete within that window

Q6: Why should you never expose stack traces in production error responses?

- Stack traces reveal your internal file structure, module names, library versions, and code logic — information attackers can use to find vulnerabilities or plan targeted attacks
- A stack trace showing at router.get (/app/routes/users.js:42) tells an attacker exactly where to look in your source code
- The safe pattern: in development, return the full error and stack. In production, only return isOperational errors with their message; return a generic 'Something went wrong' for programmer errors
- Differentiate by NODE_ENV: process.env.NODE_ENV === 'production' determines which response format to use in the global error handler

Q7: What is structured logging and why is it better than console.log?

- Structured logging emits JSON objects instead of plain text: { timestamp, level, message, requestId, userId, duration } — parseable by log aggregation tools like Datadog, Splunk, or ELK stack
- console.log output is unstructured text — you cannot easily query 'all errors from user ID 42 in the last hour' without regex parsing
- Winston, Pino, and Bunyan are the main structured logging libraries for Node.js. They support log levels (error, warn, info, debug), multiple transports (file, console, HTTP), and custom formatters
- Always include contextual metadata: logger.error('DB failed', { requestId: req.id, userId: req.user?.id, query, duration }) — makes debugging in production vastly faster

Q8: How do you handle 404 (Not Found) errors in Express?

- Add a catch-all route after all valid routes: app.all('*', (req, res, next) => next(new NotFoundError(req.originalUrl))) — it runs for any request that did not match a defined route
- This catch-all must come BEFORE the global error handler (which is always last) but AFTER all actual routes
- Return a 404 with the HTTP method and path: Cannot GET /api/v1/users/999/reviews is more helpful than a generic Not Found

- Distinguish between route not found (404 from the catch-all) and resource not found (404 thrown inside a route handler when a DB query returns nothing) — both are 404 but for different reasons

Q9: How do you handle Mongoose validation errors and duplicate key errors in the global error handler?

- Mongoose ValidationError (err.name === 'ValidationError') fires when a document fails schema validation — extract err.errors object, map to messages, return 422
- Mongoose duplicate key error (err.code === 11000) fires on unique index violation — extract err.keyValue to find which field, return 409 Conflict
- Mongoose CastError (err.name === 'CastError') fires when an invalid ID format is passed (e.g. 'abc' as a MongoDB ObjectId) — return 400 Bad Request
- Convert these to AppError instances in the global handler before sending: if (err.code === 11000) error = handleDuplicateKey(err); sendProdError(error, res)

Q10: What is graceful shutdown and how do you implement it?

- Graceful shutdown stops accepting new requests, waits for in-flight requests to complete, closes database connections, and then exits — without abruptly dropping active connections
- Steps: listen for SIGTERM/SIGINT → call server.close() (stops accepting new connections, lets existing ones finish) → close DB connections → process.exit(0)
- Set a timeout to force exit if graceful shutdown takes too long: setTimeout(() => { console.error('Forced exit'); process.exit(1); }, 30000).unref()
- PM2 and Kubernetes both send SIGTERM before forcefully killing a process — implementing graceful shutdown means zero dropped requests during rolling deploys